



# 软件架构设计

主讲：李伟刚

[liweigang@nwpu.edu.cn](mailto:liweigang@nwpu.edu.cn)

西北工业大学软件学院



# 第八章 面向服务软件架构设计

# 主要内容

- 8.1 概述
- 8.2 软件架构代理者模式
- 8.3 面向服务架构的支撑技术
- 8.4 软件架构事务模式
- 8.5 协商模式
- 8.6 服务接口设计
- 8.7 服务的协调
- 8.8 面向服务架构的集成通信图
- 8.9 服务复用



# 8.1 概述

- 面向服务架构(**Service oriented Architecture, SOA**)是一个由多个自治的服务组成的分布式软件体系结构
- 各个服务可以用不同的语言实现，运行在不同的平台上
- 服务间的通信和信息交换采用标准协议
- 对每个服务都有服务描述
- 服务消费者通过服务代理发现和调用服务（松耦合）



# 8.1 概述

## ● 服务的设计原则

### ○ 松耦合

- 服务之间相对独立

### ○ 服务契约

- 契约以一组操作的形式在服务接口中进行定义

### ○ 自治

- 服务是自包含的，能够在不需要其它服务的情况下独立运行

### ○ 抽象

- 服务的细节被隐藏起来



# 8.1 概述

## ○可复用性

- 是服务的设计目标

## ○可组合性

- 服务能够组合成更大的复合服务

## ○无状态性

- 服务几乎不保持任何关于特定客户端活动的信息

## ○可发现性

- 服务通过一种外部描述使其可以被某种发现机制所发现



## 8.2 软件架构代理者模式

- 代理者是客户端和服务的中介
- 服务向代理者注册
- 客户端通过代理者定位服务
- 当代理者建立了客户端与服务间的连接后，客户端和服务就可以直接或通过代理者进行通信了
- 代理者提供了：
  - 位置透明
  - 平台透明



## 8.2 软件架构代理者模式

- 主要模式：

- 服务注册模式
- 代理者转发模式
- 代理者句柄模式
- 服务发现模式





## 8.2 软件架构代理者模式

### ● 服务注册模式

#### ○ 服务向代理者注册服务信息

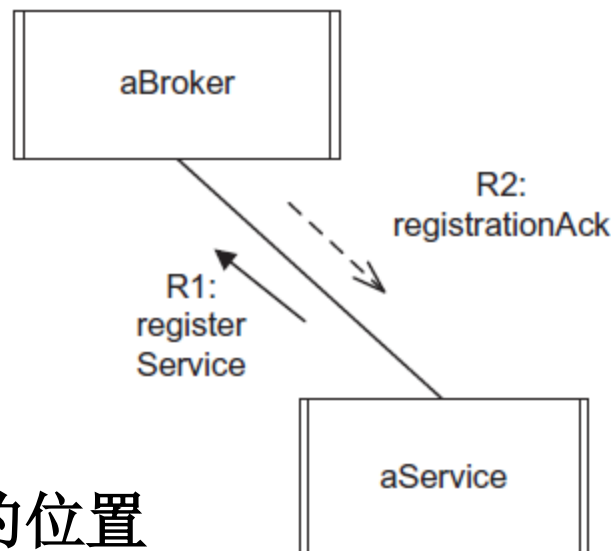
- 服务名称、服务描述和提供服务的位置

#### ○ 服务在第一次加入“代理交易所”时需要注册

#### ○ 服务每次位置变化，需重新注册

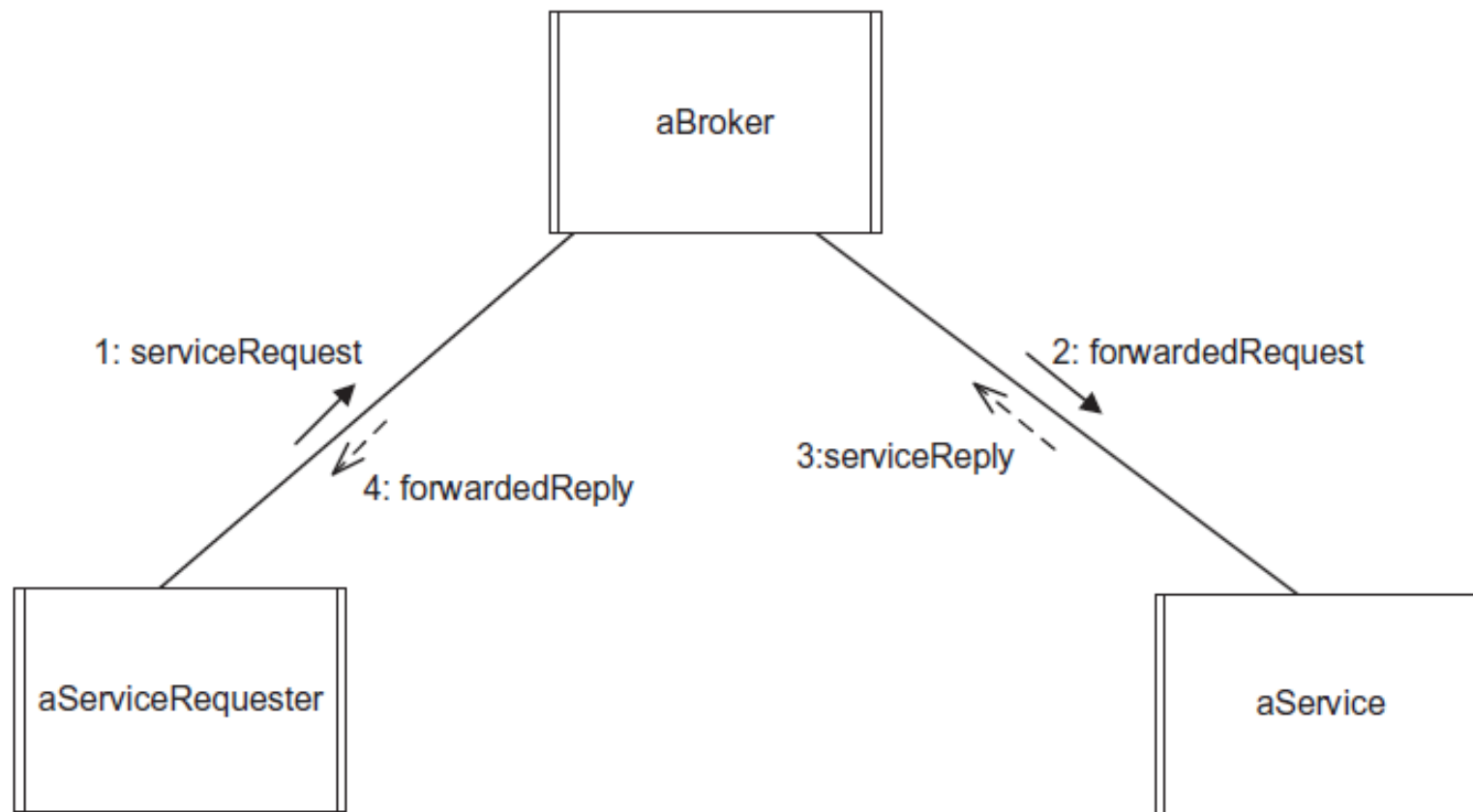
#### ○ 流程：

- 服务向代理者发送“注册服务”请求
- 代理者在**服务库**中注册服务，并向服务发送“注册确认”

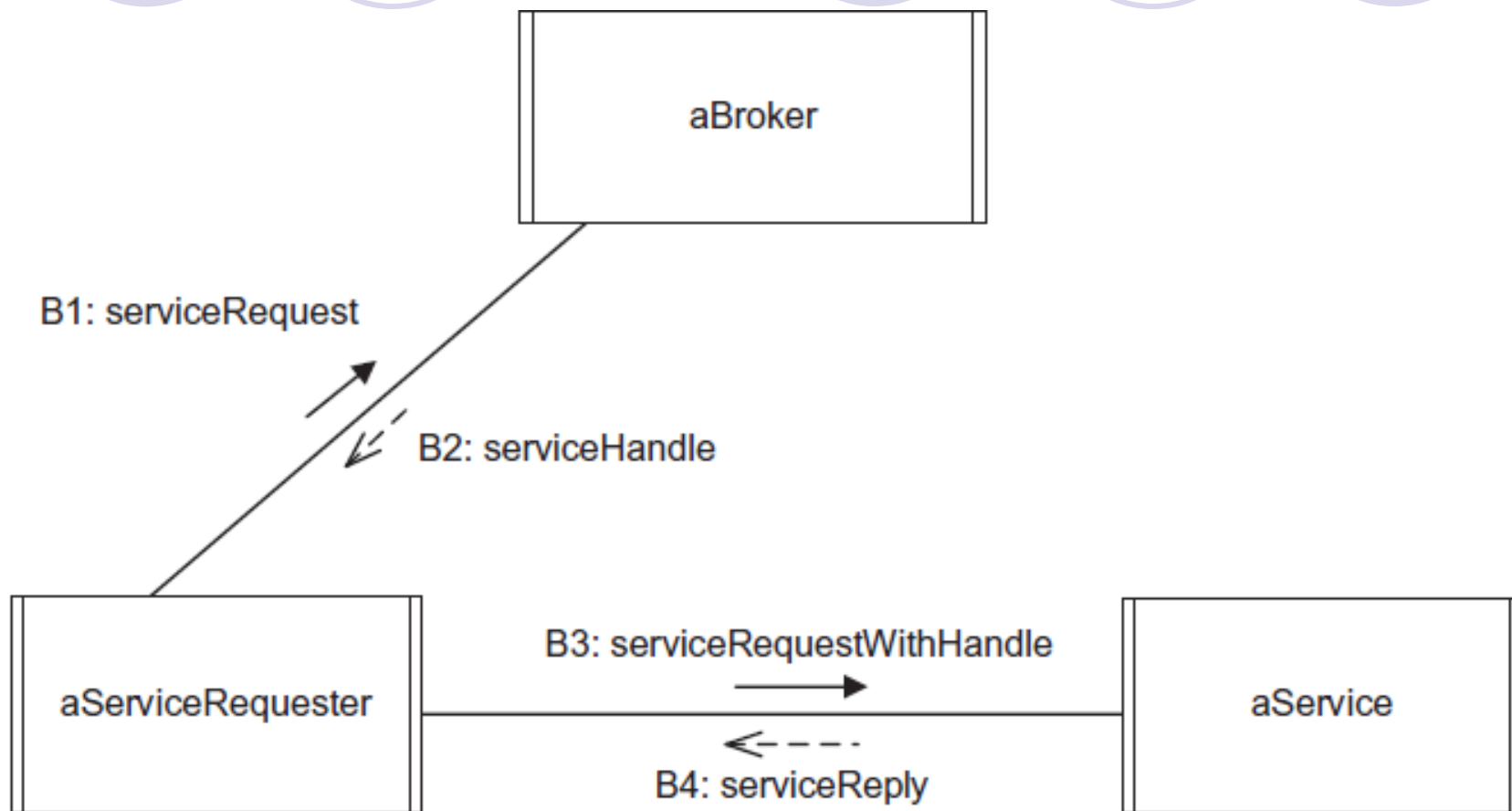


## 8.2 软件架构代理者模式

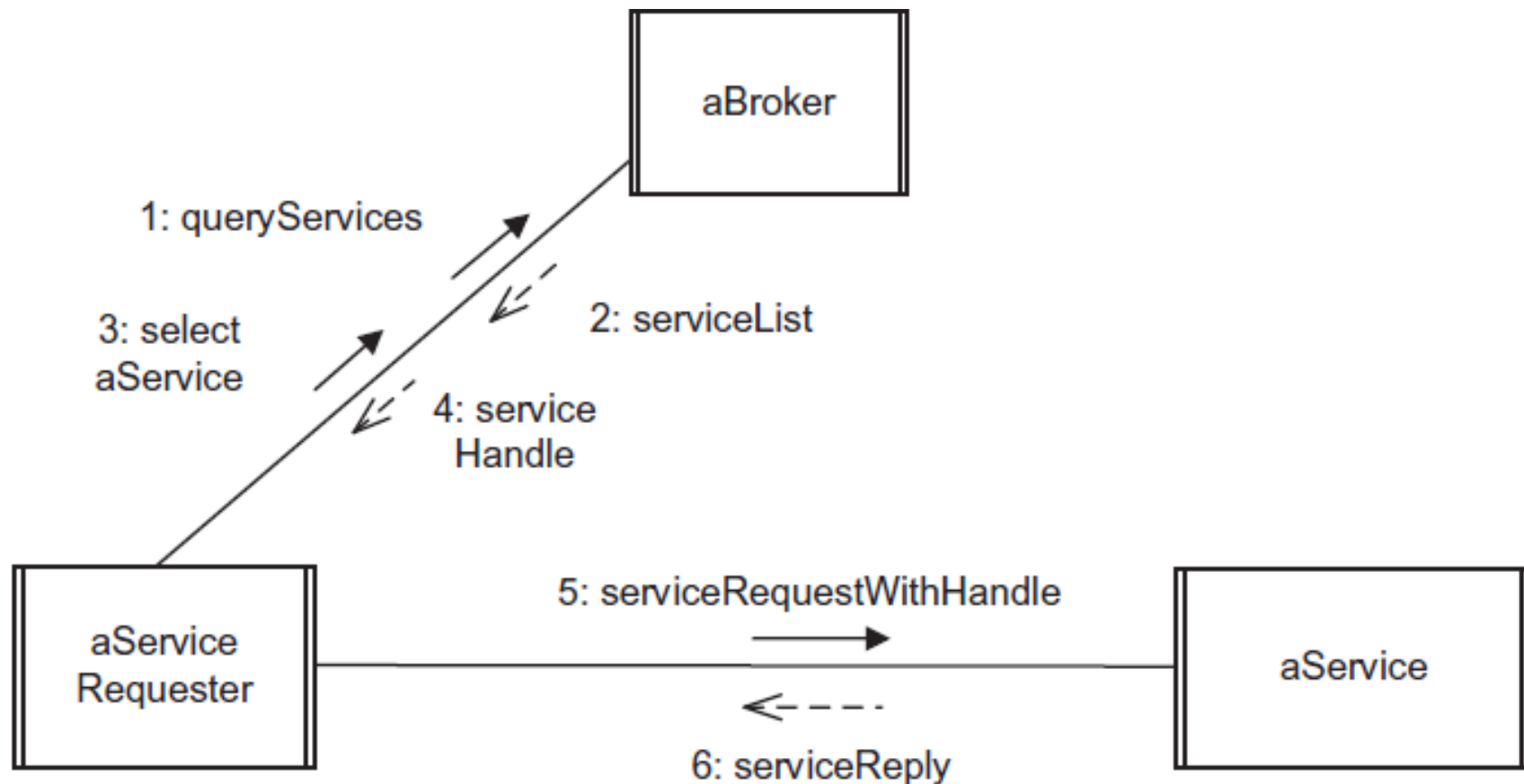
### ● 代理者转发模式



## 8.2 软件架构代理者模式



## 8.2 软件架构代理者模式



## 8.3 面向服务架构的支撑技术

### ● Web服务

- Web服务是一个使用标准的**Internet**和基于**XML**的协议访问的服务
- 为万维网上的不同软件应用间提供了标准通信手段的应用程序接口
- 它以**Internet**上的直接服务的形式为其他组织或程序提供业务功能



## 8.3 面向服务架构的支撑技术

### ● Web服务协议

- 客户端和服务间采用**SOAP**通信协议
- **SOAP**是**W3C**（万维网联盟）设计的轻量级协议，建立在**XML**和**HTTP**基础上，支持分布环境下的信息交换
- **SOAP**定义了一个发送**XML**编码数据的统一方法
  - 一个信封：描述消息内容以及消息处理方式
  - 一组编码规则：描述特定应用的数据类型
  - 一个描述**RPC**和响应的规约



## 8.3 面向服务架构的支撑技术

### ● 注册服务

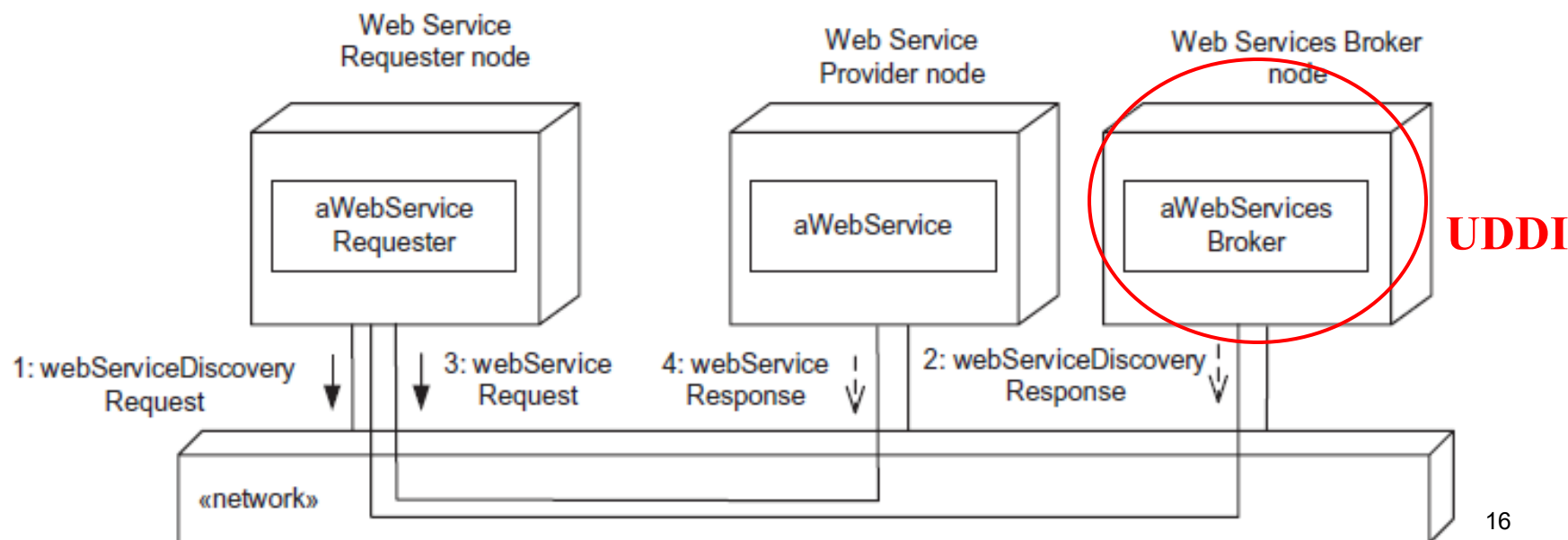
- 用于使服务能够被其客户端找到
- 服务靠注册服务 **发布**或**注册**
- 大多数代理者提供了一个注册服务
  - 如，**CORBA**、**Web Services**代理者
- **Web**服务使用 **服务库**来支持基于**Web**的服务发布和定位
- 服务提供者将服务和服务描述注册到服务库中
- 客户端在服务库中查找合适的服务



## 8.3 面向服务架构的支撑技术

### ● 代理和发现服务

- 代理是客户端和服务之间交互的中介
- **Web**服务代理者是一种典型的代理
- **Web**服务的信息通过**UDDI**（统一描述、发现和集成）框架来定义





## 8.4 软件架构事务模式

- 事务是客户端对由两个或更多操作组成的服务的请求，这些操作属于同一个逻辑功能并必须全部完成或全部不做
- 事务在客户端生成并发送给服务处理
- 事务的**ACID**特性
  - **A**: 原子性，事务不可分割
  - **C**: 一致性，事务执行完，系统处于一致状态
  - **I**: 隔离性，两个事务间不相互影响
  - **D**: 持续性，事务结果的持久性



## 8.4 软件架构事务模式

- 两阶段提交协议模式

- 用于在分布式系统中处理事务的原子性问题

- 例如，跨行转账

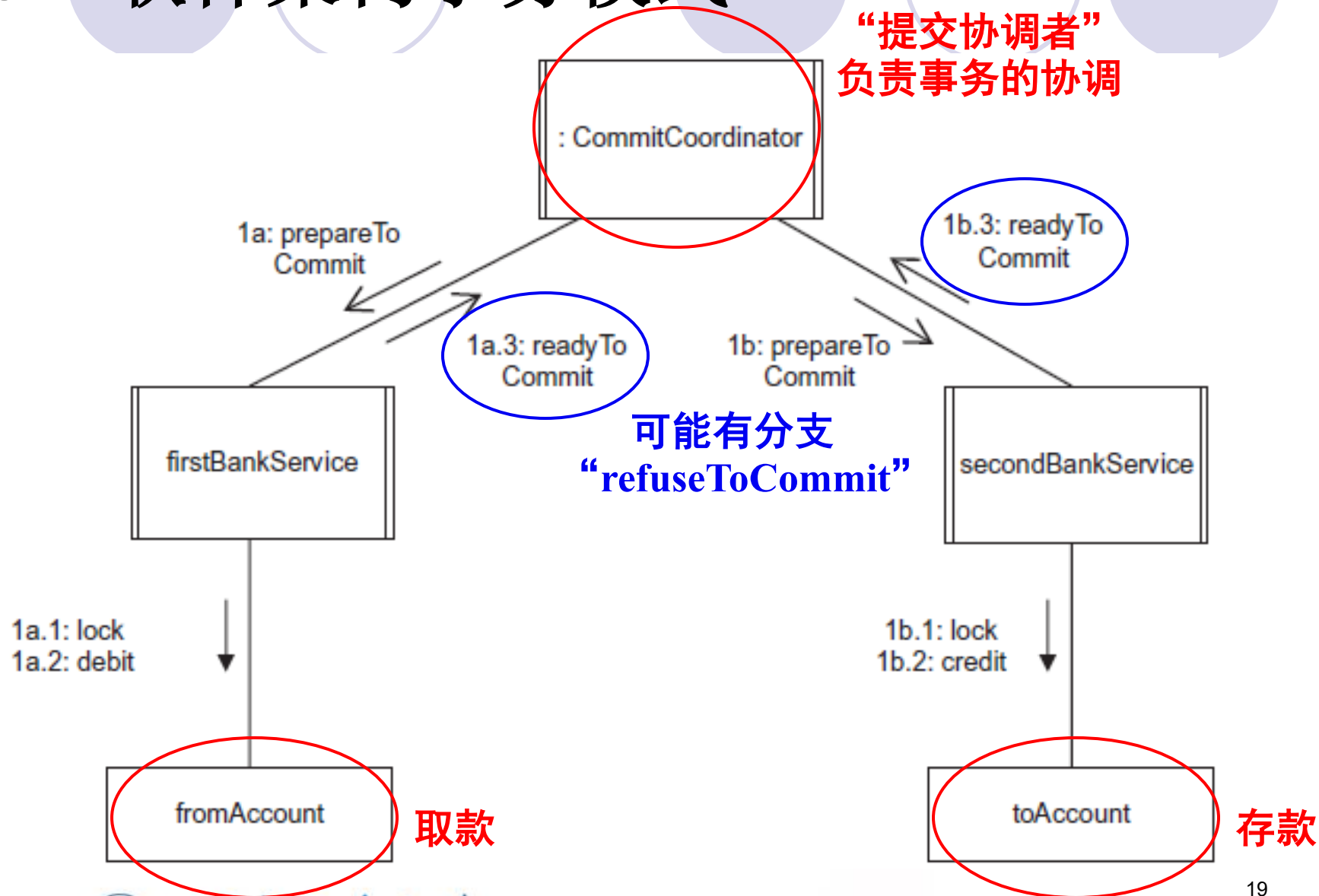
- 转账事务由 **借** 和 **贷** 两个原子操作组成

- 转账事务要么 **被提交**，要么 **被终止**

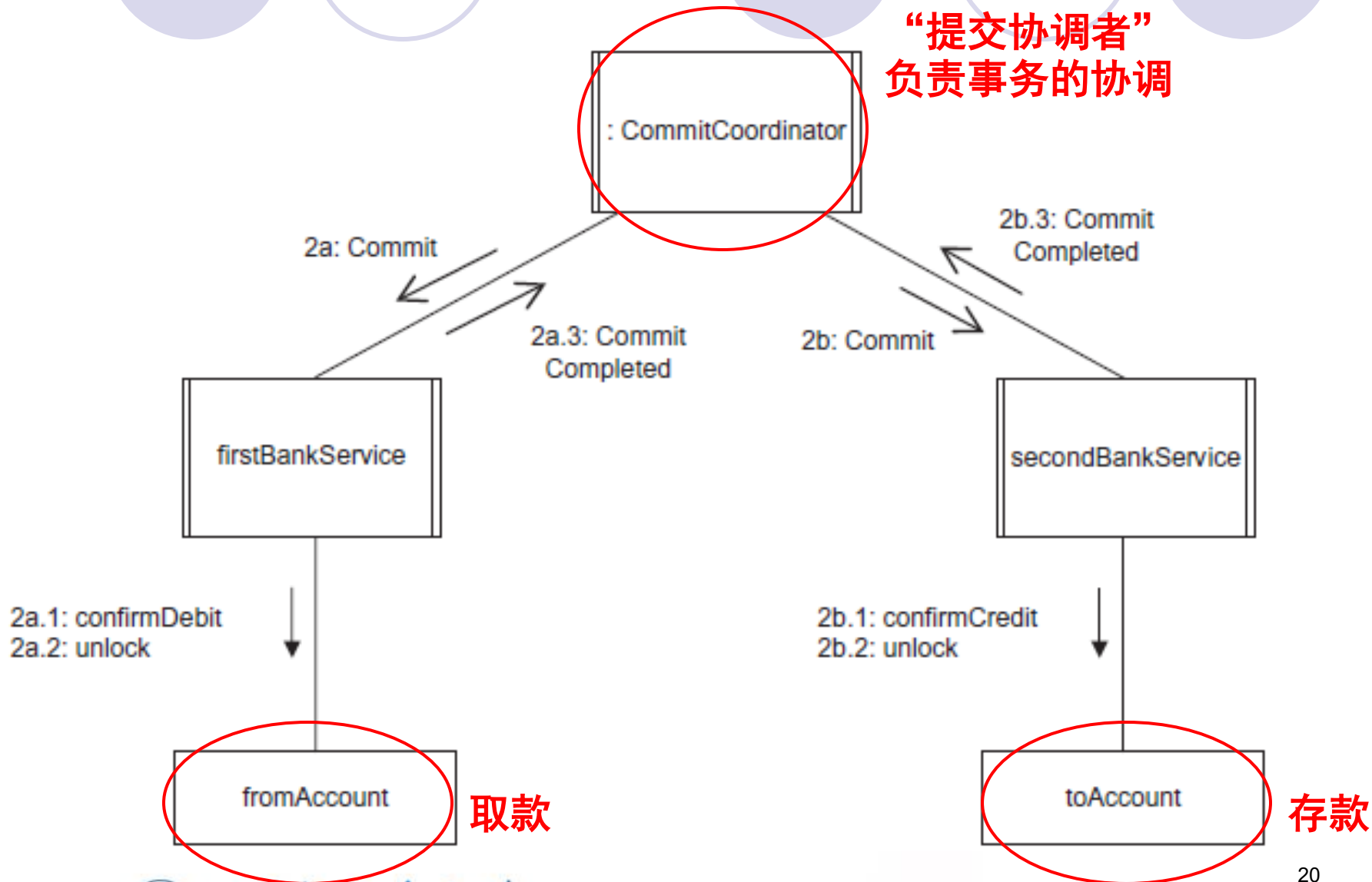
- 第一阶段通信图



## 8.4 软件架构事务模式



## 8.4 软件架构事务模式



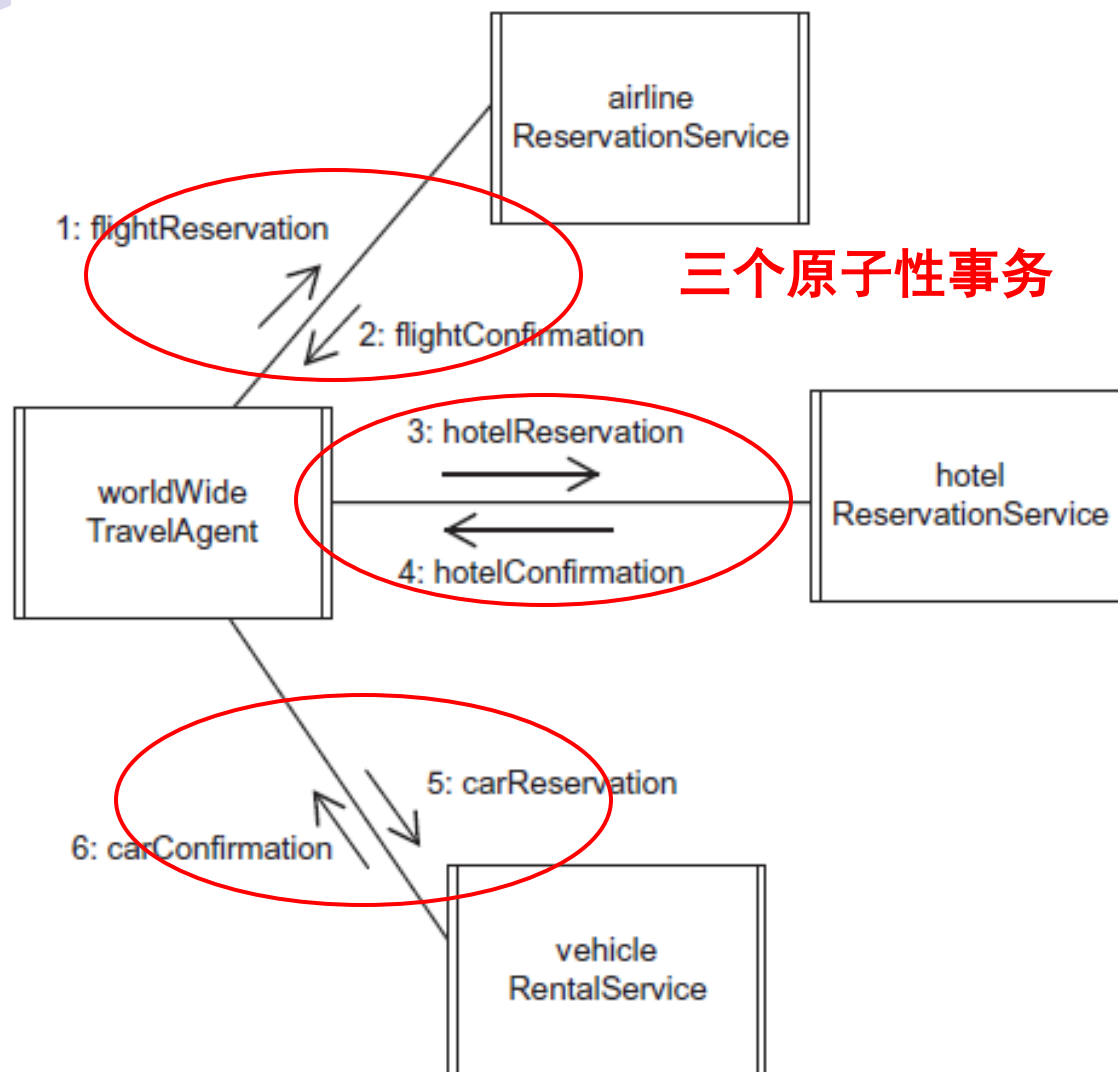
## 8.4 软件架构事务模式

- 复合事务模式

- 需要部分回滚
- 当客户端的事务需求能够被分解成更小的原子性事务，且每个原子性事务能够被单独执行或回滚时，可以使用此模式
- 例如，旅行社
  - 预定机票
  - 预定宾馆
  - 租车



## 8.4 软件架构事务模式



## 8.4 软件架构事务模式

### ● 长事务模式

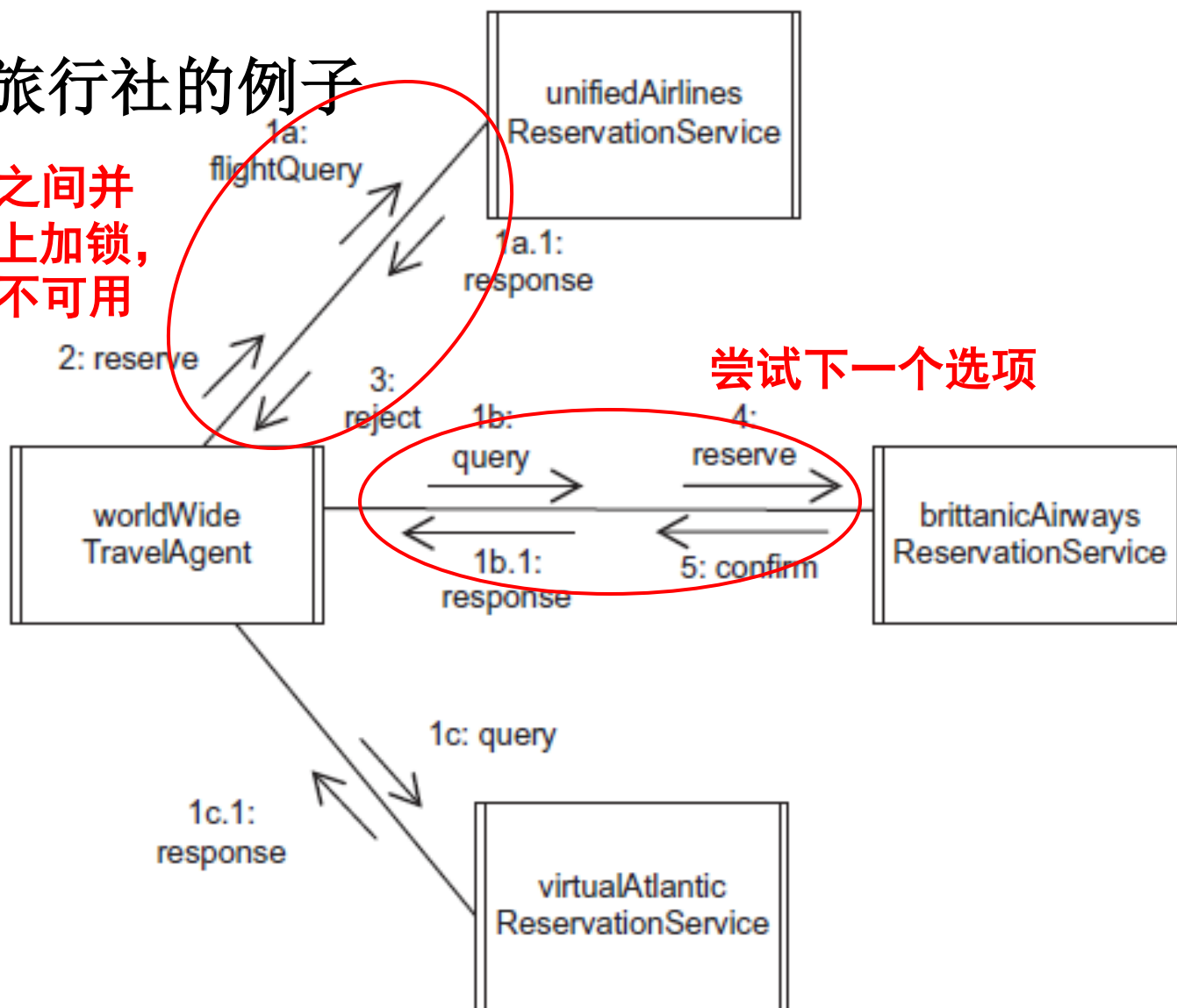
- 是有人参与、需要很长甚至无限长的时间来执行的事务
- 基于人行为的不可预知性
- 将长事务分解成两个或更多的单独的事务（通常是两个）
- 人的决策发生在连续的事务对儿之间



## 8.4 软件架构事务模式

### ○ 旅行社的例子

两次请求之间并  
未在记录上加锁，  
预定变得不可用



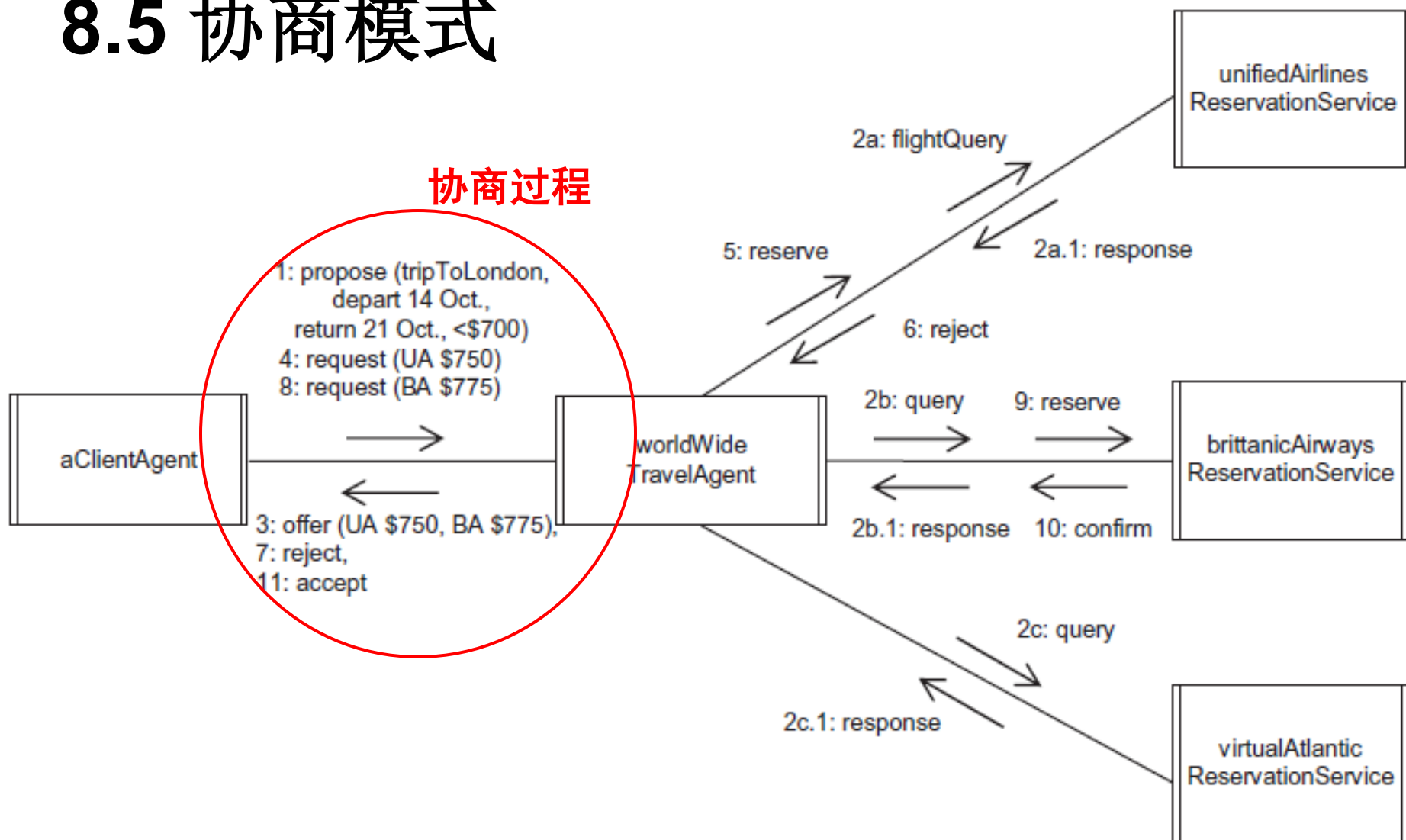


## 8.5 协商模式

- 软件主体（**agent**）之间协商，共同做出决策
- 分为**客户主体**和**服务主体**
  - 一个客户主体代表用户向一个服务主体发起一个服务提议
  - 服务主体尝试满足提议，可能涉及与其他服务的通信
  - 确定了可用的选项后，服务主体向客户主体提出一个或多个与客户主体提议最接近的选项
  - 客户主体可以选择向其中一个选项发出请求、提出更多的选项、或者拒绝
  - 若服务主体能满足客户主体的请求，则接受请求，否则就会拒绝



## 8.5 协商模式



## 8.6 服务接口设计

- 通过客户端与服务对象之间的交互确定服务操作
- 通过分析交互消息确定操作名称、输入输出参数



## 8.7 服务的协调

- 涉及多个服务的**SOA**应用通常需要在这些服务间进行协调
- 将协调的实现细节和各个服务的功能分离可以确保服务间的松耦合
- 通过协调机制来控制各个服务的访问顺序
- 协调机制包括：
  - 编曲(**orchestration**): 集中式过程逻辑
  - 编舞(**choreography**): 分布式协调, 用于跨组织协调
- 确定协调者的接口 (方法同前)



## 8.8 面向服务架构的集成通信图

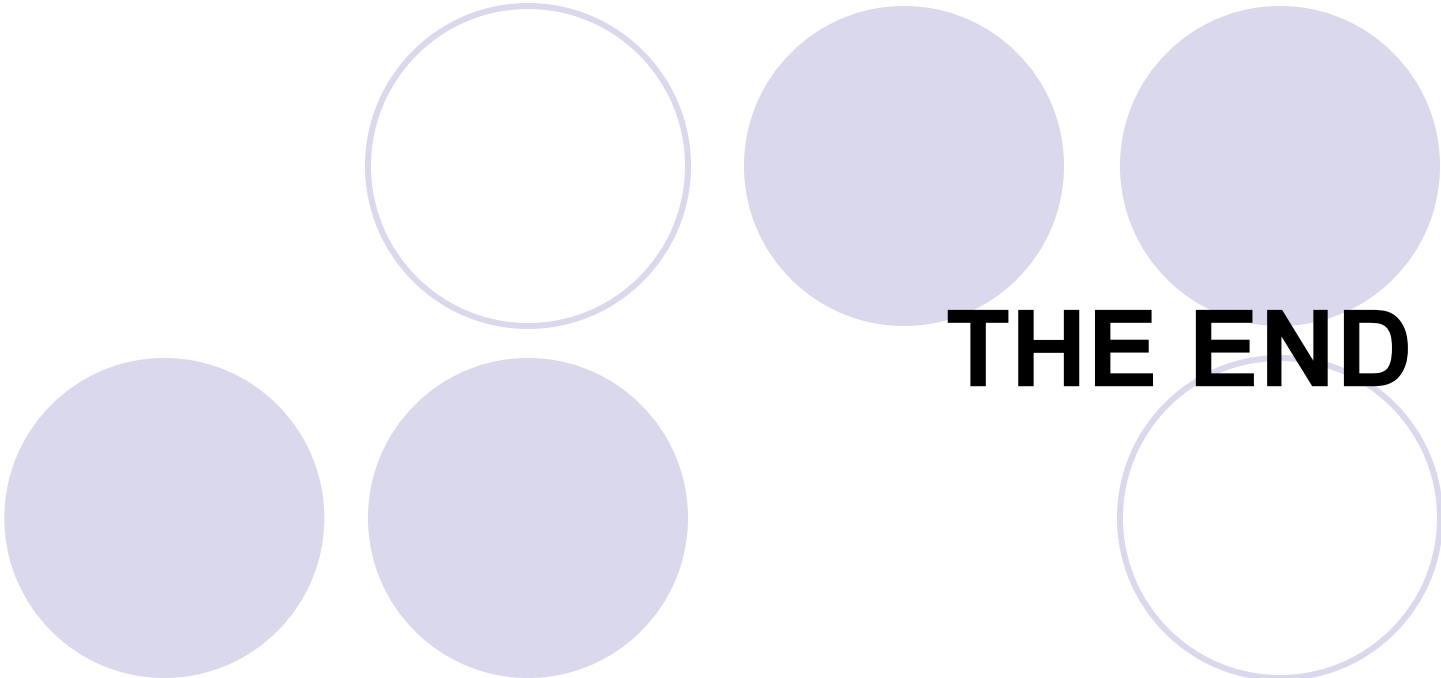
- 确定了服务和协调者的接口后，就可以开发集成通信图
- 对**SOA**来说，集成通信图既是并发的又是分布的
- 通信模式：
  - 与服务的通信一般为同步通信，**请求/响应**
  - 亦可以使用带回调的异步通信
  - 协调者之间常使用异步通信



## 8.9 服务复用

- 服务一般只有供给接口，而没有请求接口（除了使用带回调的异步通信），这样使服务更独立，便于复用
- 所有被描述的服务都可以用于不同的面向服务的应用中
- 创建新的协调者对象，而充分复用其他服务



Five light purple circles are arranged in a pentagonal pattern around the text. Two are solid, and three are hollow outlines.

**THE END**